

Pigs: Protocol-Native Phased Orchestration for LLM API Requests

Anonymous

Abstract

Large Language Model (LLM) agents typically interact with providers through stateless API calls, lacking built-in mechanisms for planning, execution verification, and failure recovery. While frameworks like ReAct and Reflexion add multi-stage reasoning within a single conversation, they rely on the LLM itself to manage control flow—an approach that is fragile, protocol-specific, and opaque to downstream tooling. We present **Pigs**, a phased orchestration middleware that wraps LLM API requests with a deterministic Pre→Executor→Post state machine using lightweight text control markers (PIGEND/PIGFAIL) for routing. Pigs preserves the complete protocol-native HTTP request—method, path, headers, and JSON body—across three API protocols (OpenAI Chat Completions, Anthropic Messages, OpenAI Responses), modifying only the model identifier, current user input, and phase transcript. Phase subrequests re-enter the local proxy via HTTP loopback, transparently inheriting channel selection, model mapping, body cleaning, and retry logic. External tool execution is supported through a bounded in-memory continuation store with TTL and capacity limits. We evaluate Pigs through four experiments: an ablation study isolating each phase’s contribution, a control marker effectiveness study, a three-protocol consistency evaluation, and a cost-benefit analysis. **Experimental results are pending and will be filled upon completion of the evaluation.**

1 Introduction

Modern LLM-based agents interact with model providers through HTTP APIs that are fundamentally stateless: a client sends messages and receives a completion, with no built-in notion of planning, execution verification, or failure recovery. To achieve complex multi-step behavior, practitioners have developed orchestration frameworks—from simple chain-of-thought prompting [Wei et al., 2022] to interleaved reasoning-acting loops [Yao et al., 2023b] and reflective self-improvement [Shinn et al., 2023, Madaan et al., 2023]. These approaches share a common limitation: they place the LLM in charge of its own control flow. The model must decide when to plan, when to execute, and when to reflect, using natural language cues that are inherently ambiguous and error-prone. 现代基于大语言模型（LLM）的智能体通过本质上无状态的 HTTP API 与模型提供商交互：客户端发送消息并接收补全结果，而系统本身不具备规划、执行验证或故障恢复的内在机制。为实现复杂的多步骤行为，实践者开发了多种编排框架——从简单的思维链提示 [Wei et al., 2022]，到推理与行动交替的循环 [Yao et al., 2023b]，再到反思式自我改进 [Shinn et al., 2023, Madaan et al., 2023]。这些方法存在一个共同的局限性：它们将 LLM 置于自身控制流的主导地位。模型必须自行决定何时规划、何时执行、何时反思，而所依赖的自然语言线索本质上具有模糊性且容易出错。

This design creates three problems. First, **control-flow fragility**: if the model deviates from the expected reasoning pattern (e.g., skipping the planning phase or prematurely declaring completion), the entire agent loop breaks. Second, **protocol coupling**: most orchestration frameworks are tightly coupled to a single API protocol (typically OpenAI Chat Completions), requiring reimplementations when switching providers. Third, **opacity**: external tooling (proxies, observability platforms, cost monitors) cannot inspect or influence the orchestration logic because it is embedded inside the agent’s prompt engineering rather than exposed as a system-level concern. 这一设计导致了三个问题。第一，**控制流脆弱性**：如果模型偏离了预期的推理模式（例如跳过规划阶段或过早宣布完成），整个智能体循环便会中断。第二，**协议耦合**：大多数编排框架与单一 API 协议（通常为 OpenAI Chat Completions）紧密耦合，在切换提供商时需要重新实现。第三，**不透明性**：外部工具（代理、可观测性平台、成本监控器）无法检查或影响编排逻辑，因为该逻辑内嵌于智能体的提示工程中，而非作为系统层面的关注点暴露出来。

We argue that LLM orchestration should be elevated from a prompting technique to a *middleware concern*—a transparent layer between the client and the provider that enforces a deterministic control-flow structure while leaving semantic decisions to the model. This separation mirrors the classical middleware pattern in distributed systems, where transport-level concerns (routing, retry, load balancing)

are handled outside the application logic. 我们认为, LLM 编排应当从一种提示技术提升为一种中间件关注点——一个位于客户端与提供商之间的透明层, 在执行确定性的控制流结构的同时, 将语义决策交由模型处理。这种分离借鉴了分布式系统中经典的中间件模式, 其中传输层面的问题 (路由、重试、负载均衡) 在应用逻辑之外处理。

We present **Pigs**, a phased orchestration middleware that implements this vision. Pigs wraps each LLM API request with a three-phase state machine: 我们提出了 **Pigs**, 一种分阶段编排中间件, 实现了上述愿景。Pigs 以三阶段状态机封装每个 LLM API 请求:

1. **Pre** (planning): the model analyzes the task, identifies required information, and formulates an execution plan. If the task is trivially simple, the model may short-circuit with a **PIGEND** marker. **Pre** (规划): 模型分析任务, 识别所需信息, 并制定执行计划。若任务极其简单, 模型可通过 **PIGEND** 标记直接短路。
2. **Executor** (execution): the model executes the plan, gathering information and producing output. Tool calls are returned to the external agent for execution. **Executor** (执行): 模型执行计划, 收集信息并产生输出。工具调用被返回给外部智能体执行。
3. **Post** (review): the model independently reviews the executor’s output against the goal. It may confirm completion (**PIGEND**), signal failure (**PIGFAIL**), or continue working (no marker). **Post** (审查): 模型独立地根据目标审查执行器的输出。它可以确认完成 (**PIGEND**)、发出失败信号 (**PIGFAIL**), 或继续工作 (无标记)。

The state machine is driven by *control markers*—short text tokens (**PIGEND**, **PIGFAIL**) that the model appends to its output. These markers are detected by the middleware, not the model’s own reasoning; they serve as routing signals, not as content. Crucially, the middleware strips these markers from the visible output, so the client sees only the model’s substantive text. Unlike ReAct, where the model may skip planning or reflection, Pigs *guarantees* that every non-trivial request passes through all three phases—the state machine cannot be bypassed by the model. 该状态机由控制标记驱动——模型附加到其输出中的简短文本标记 (**PIGEND**、**PIGFAIL**)。这些标记由中间件检测, 而非由模型自身的推理检测; 它们充当路由信号, 而非内容。关键在于, 中间件会从可见输出中剥离这些标记, 因此客户端只能看到模型的实质文本。与 ReAct 不同——在 ReAct 中模型可能跳过规划或反思——Pigs 保证每个非平凡请求都会经过全部三个阶段: 该状态机无法被模型绕过。

Pigs makes six key contributions: Pigs 做出了六项关键贡献:

1. **Phased orchestration via text control markers**: a deterministic Pre→Executor→Post state machine that uses lightweight text markers for routing, without injecting orchestration-specific tools or system prompts into the model’s context. **基于文本控制标记的分阶段编排**: 一种确定性的 Pre→Executor→Post 状态机, 使用轻量级文本标记进行路由, 无需向模型上下文中注入编排特定的工具或系统提示。
2. **Protocol-native request preservation**: the complete HTTP request—method, path/query, headers, and JSON body—is preserved across three API protocols, with only the model identifier, current user input, and phase transcript modified. Unknown fields, media blocks, and provider extensions pass through untouched. **协议原生的请求保留**: 完整的 HTTP 请求——方法、路径/查询、头部和 JSON 主体——在三种 API 协议间保持不变, 仅修改模型标识符、当前用户输入和阶段记录。未知字段、媒体块和提供商扩展均原样透传。
3. **HTTP loopback transport**: phase subrequests re-enter the local proxy via HTTP, transparently inheriting channel selection, model mapping, body cleaning, thinking-effort injection, and retry logic—without requiring a separate in-process dispatch path. **HTTP 回环传输**: 阶段子请求通过 HTTP 重新进入本地代理, 透明地继承通道选择、模型映射、主体清洗、思考力度注入和重试逻辑——无需单独的进程内分发路径。
4. **Bounded in-memory continuation**: external tool execution is supported through a continuation store with TTL, capacity limits, and tombstone-based error reporting, never persisting authentication headers to disk. **有界内存续传**: 通过具有 TTL、容量限制和基于墓碑的错误报告的续传存储来支持外部工具执行, 且绝不将认证头部持久化到磁盘。
5. **Ablation study** isolating the contribution of each phase (Pre, Executor, Post) to overall task performance on public benchmarks (SWE-bench, GAIA). **消融研究**, 在公开基准测试 (SWE-bench、GAIA) 上隔离各阶段 (Pre、Executor、Post) 对整体任务性能的贡献。
6. **Control marker effectiveness study** showing whether **PIGFAIL**-based failure recovery improves outcomes, plus a **three-protocol consistency evaluation** and **cost-benefit analysis** quantifying the token, latency, and API call overhead of orchestration. **控制标记有效性研究**, 验证基于

PIGFAIL 的故障恢复是否能改善结果，以及三协议一致性评估和成本效益分析，量化编排的令牌、延迟和 API 调用开销。

The remainder of this paper describes the architecture (Section 3), implementation (Section 4), experimental evaluation (Section 5), related work (Section 7), and design discussion (Section 8). 本文的其余部分描述了架构 (Section 3)、实现 (Section 4)、实验评估 (Section 5)、相关工作 (Section 7) 和设计讨论 (Section 8)。

2 Background and Motivation

2.1 LLM API Protocols

Three API protocols dominate the LLM provider landscape: 三种 API 协议主导了 LLM 提供商生态：

- **OpenAI Chat Completions** (`/chat/completions`): messages with roles (system, user, assistant, tool), tool calls, and streaming via Server-Sent Events (SSE) terminated by `[DONE]`. **OpenAI Chat Completions** (`/chat/completions`): 带有角色 (system、user、assistant、tool) 的消息、工具调用，以及通过 Server-Sent Events (SSE) 进行的流式传输，以 `[DONE]` 终止。
- **Anthropic Messages** (`/v1/messages`): content blocks (text, tool_use, tool_result, thinking), system as a top-level field, and SSE terminated by `message_stop`. **Anthropic Messages** (`/v1/messages`): 内容块 (text、tool_use、tool_result、thinking)，system 作为顶层字段，SSE 以 `message_stop` 终止。
- **OpenAI Responses** (`/responses`): a unified input/output array with typed items (message, reasoning, function_call, function_call_output), instructions as a top-level field, and SSE terminated by `response.completed`. **OpenAI Responses** (`/responses`): 具有类型化条目 (message、reasoning、function_call、function_call_output) 的统一输入/输出数组，instructions 作为顶层字段，SSE 以 `response.completed` 终止。

Each protocol has distinct message structure, tool-call representation, streaming format, and termination semantics. A middleware that supports all three must either (a) convert between protocols (lossy) or (b) preserve each protocol’s native format (lossless). Pigs chooses the latter. 每种协议具有不同的消息结构、工具调用表示、流式格式和终止语义。一个同时支持三种协议的中间件要么 (a) 在协议间进行转换 (有损)，要么 (b) 保留每种协议的原生格式 (无损)。Pigs 选择了后者。

2.2 Multi-Stage LLM Orchestration

Several lines of work add structure to LLM interactions: 若干研究方向为 LLM 交互增加了结构：

ReAct [Yao et al., 2023b] interleaves reasoning traces and actions in a single LLM output. The model decides when to think and when to act, with no external control flow. This is elegant but fragile: the model may skip planning, act prematurely, or fail to reflect on errors. **ReAct** [Yao et al., 2023b] 在单一的 LLM 输出中交替进行推理轨迹和行动。模型自行决定何时思考、何时行动，没有外部控制流。这种方式优雅但脆弱：模型可能跳过规划、过早行动或未能反思错误。

Reflexion [Shinn et al., 2023] adds a reflection loop where the model receives verbal feedback and stores it in episodic memory for the next attempt. While effective, the reflection trigger is model-driven—there is no guarantee the model will reflect at the right time. **Reflexion** [Shinn et al., 2023] 增加了反思循环，模型接收语言反馈并将其存入情景记忆中供下次尝试使用。虽然有效，但反思的触发是模型驱动的——无法保证模型会在正确的时机进行反思。

Self-Refine [Madaan et al., 2023] uses a single LLM as generator, critic, and refiner. The control flow is implicit in the prompt structure, and the model may deviate from the expected generate-critic-refine sequence. **Self-Refine** [Madaan et al., 2023] 使用单个 LLM 同时充当生成器、批评器和精炼器。控制流隐含在提示结构中，模型可能偏离预期的“生成-批评-精炼”序列。

Plan-and-Solve [Wang et al., 2023b] explicitly separates planning from execution in a two-phase prompt. However, both phases occur within a single LLM call, and there is no external verification of whether the plan was followed. **Plan-and-Solve** [Wang et al., 2023b] 在两阶段提示中明确将规划与执行分离。然而，两个阶段均发生在单次 LLM 调用中，且没有对计划是否被执行的外部验证。

These approaches share a common pattern: they embed control flow *inside* the prompt, relying on the model’s instruction-following ability to maintain phase discipline. Pigs inverts this: control flow is *external* to the model, enforced by a deterministic state machine, while the model contributes only semantic content and routing signals. 这些方法共享一个模式：它们将控制流内嵌于提示中，依赖模型的指令遵循能力来维持阶段纪律。Pigs 颠倒了这一做法：控制流外置于模型之外，由确定性状态机执行，而模型仅贡献语义内容和路由信号。

2.3 LLM Middleware

The concept of LLM middleware—a transparent layer between client and provider—has been discussed in position papers [Dzanic et al., 2024] but remains under-explored in terms of implemented systems. Existing proxy-like systems (e.g., LiteLLM, OpenRouter) focus on model routing and cost optimization, not on orchestration logic. TokenMizer [Liu, 2026] adds session state management as a proxy feature, but does not impose phase structure. LLM 中间件的概念——一个位于客户端与提供商之间的透明层——已在立场论文中被讨论 [Dzanic et al., 2024]，但在实现系统层面仍探索不足。现有的类代理系统（如 LiteLLM、OpenRouter）专注于模型路由和成本优化，而非编排逻辑。TokenMizer [Liu, 2026] 将会话状态管理作为代理功能添加，但不施加阶段结构。

Pigs occupies a distinct niche: it is both a middleware (transparent, protocol-preserving, non-invasive) and an orchestrator (enforcing a phase structure with control markers). No prior system combines these two roles. Pigs 占据了一个独特的定位：它既是中间件（透明、协议保留、非侵入式），又是编排器（通过控制标记强制执行阶段结构）。此前没有系统将这两种角色结合在一起。

3 System Design

Pigs is structured around two core layers: a *proxy* layer that routes HTTP requests and a *phased runtime* layer that orchestrates the Pre→Executor→Post state machine. A general-purpose agent client connects to the proxy via standard HTTP, using either `{model}` (passthrough) or `{model}-pig` (phased orchestration) as the model identifier. This section describes the core design of the proxy and runtime layers. Pigs 围绕两个核心层构建：负责路由 HTTP 请求的代理层和负责编排 Pre→Executor→Post 状态机的分阶段运行时层。通用 Agent 客户端通过标准 HTTP 连接代理，使用 `{model}`（透传）或 `{model}-pig`（相位编排）作为模型标识符。本节描述代理和运行时层的核心设计。

3.1 Architecture Overview

The system operates as a single-process server listening on one port (default 3927). All three API protocols are served from the same port, distinguished by request path: 系统作为单进程服务器运行，监听一个端口（默认 3927）。三种 API 协议均从同一端口提供服务，通过请求路径区分：

- `/chat/completions` → OpenAI Chat `/chat/completions` → OpenAI Chat
- `/v1/messages` → Anthropic Messages `/v1/messages` → Anthropic Messages
- `/responses` → OpenAI Responses `/responses` → OpenAI Responses

Requests are routed by model identifier: models without the `-pig` suffix are passed through to the upstream provider (with retry, body cleaning, and thinking-effort injection); models with the `-pig` suffix enter the phased runtime. Each configured model is automatically exposed as two identifiers: `{model}` (passthrough) and `{model}-pig` (phased). 请求通过模型标识符路由：不带 `-pig` 后缀的模型直接透传至上游提供商（附带重试、主体清洗和思考力度注入）；带 `-pig` 后缀的模型进入分阶段运行时。每个配置模型自动暴露为两个标识符：`{model}`（透传）和 `{model}-pig`（分阶段）。

Figure 1 illustrates the request flow. When a `-pig` request arrives, the proxy constructs a `HttpRequestEnvelope` and hands it to the `HttpPhasedRuntime`. Each phase subrequest is sent back to the proxy via HTTP loopback (with an internal token header), re-entering the passthrough path with the `-pig` suffix removed. This design ensures that phase subrequests transparently inherit all proxy-level features. Figure 1 展示了请求流。当 `-pig` 请求到达时，代理构建一个 `HttpRequestEnvelope` 并将其交给 `HttpPhasedRuntime`。每个阶段子请求通过 HTTP 回环（附带一个内部令牌头部）发送回代理，在去除 `-pig` 后缀后重新进入透传路径。这一设计确保阶段子请求透明地继承所有代理级特性。

3.2 Phase State Machine

The core of Pigs is a pure state machine with three phases and three transition signals. The state is encapsulated in `OrchestrationState`, which tracks the current phase, accumulated outputs, failure history, and iteration counters. Pigs 的核心是一个具有三个阶段和三个转移信号的纯状态机。状态封装在 `OrchestrationState` 中，用于跟踪当前阶段、累积输出、失败历史和迭代计数器。

The state machine has several key properties: 该状态机具有若干关键特性：

Markerless Post stays in Post. When the Post phase produces output without a control marker, the state machine continues in Post (not back to Executor). This design choice prevents infinite Executor↔Post loops: the model is expected to make incremental progress within Post until it either completes or fails. 无标记的 Post 停留在 Post。当 Post 阶段产生不含控制标记的输出时，状态机继续停留在

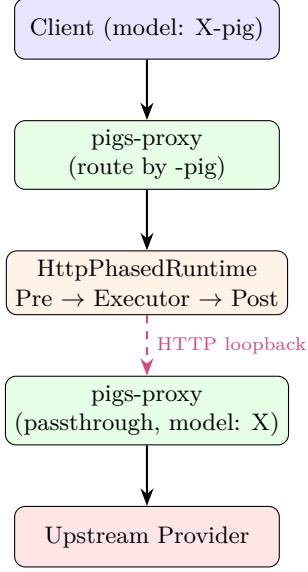


Figure 1: Request flow for a `-pig` model. Phase subrequests re-enter the proxy via HTTP loopback (dashed), inheriting passthrough features.

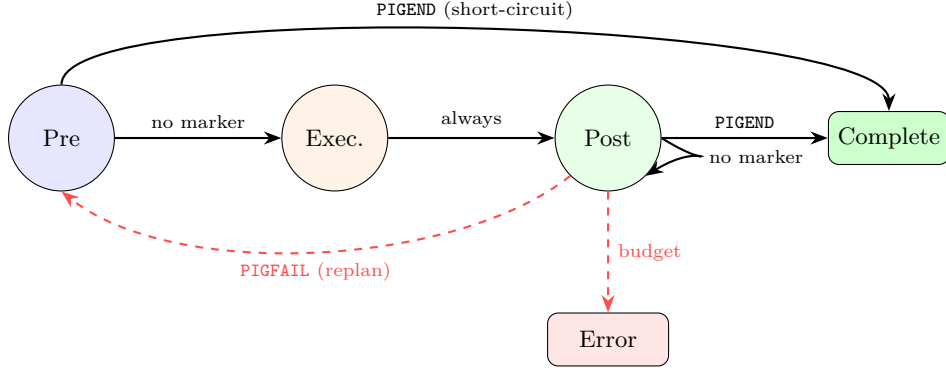


Figure 2: Phase state machine transitions. Solid arrows are forward transitions; dashed red arrows are failure and budget-exhaustion paths. The curved arrow from Post back to Pre represents PIGFAIL-triggered replanning. The self-loop on Post (right side) represents markerless continuation.

Post 阶段（而非返回 Executor）。这一设计选择防止了 `Executor ↔ Post` 的无限循环：模型应在 Post 内部逐步推进，直至完成或失败。

Budget exhaustion is an error, not success. When the Post iteration budget is exhausted, the state machine returns an `OrchestrationError`, not a successful completion. This prevents the system from silently accepting incomplete work. **预算耗尽即为错误，而非成功。** 当 Post 迭代预算耗尽时，状态机返回 `OrchestrationError`，而非成功完成。这防止了系统静默接受不完整的工作。

Failure preserves history. When PIGFAIL triggers a replan, the failure output is preserved and presented to the next Pre phase as numbered failure records (“1st failure: ...”, “2nd failure: ...”). This allows the model to learn from previous attempts. **失败保留历史。** 当 PIGFAIL 触发重新规划时，失败输出会被保留并以编号的失败记录形式呈现给下一个 Pre 阶段（“第 1 次失败: ...”、“第 2 次失败: ...”）。这使得模型能够从先前的尝试中学习。

Figure 2 illustrates the complete state machine. The Pre phase can short-circuit to Complete (trivially simple tasks), and the Post phase has a self-loop for markerless continuation. Figure 2 展示了完整的状态机。Pre 阶段可短路至 Complete（极其简单的任务），Post 阶段具有用于无标记继续的自环。

3.3 Control Markers

Control markers are short text tokens that the model appends to its output to signal phase transitions: 控制标记是模型附加到其输出中的简短文本标记，用于发出阶段转移信号：

Algorithm 1 Phase state machine transition

```
1: function ADVANCE(output)
2:   phase  $\leftarrow$  current phase
3:   if phase = Pre then
4:     if detect(output) = PIGEND then
5:       return Complete ▷ Short-circuit: trivially simple task
6:     else if detect(output) = PIGFAIL then
7:       return replan(output) ▷ Failure: return to Pre
8:     else
9:       pre_output  $\leftarrow$  strip(output)
10:      return Continue(Executor)
11:    end if
12:  else if phase = Executor then
13:    executor_outputs  $\leftarrow$  strip(output)
14:    return Continue(Post) ▷ Always proceed to Post
15:  else ▷ phase = Post
16:    if detect(output) = PIGEND then
17:      return Complete
18:    else if detect(output) = PIGFAIL then
19:      return replan(output)
20:    else
21:      if post_iterations  $\geq$  max then
22:        return Error(PostBudgetExceeded)
23:      end if
24:      post_iterations += 1
25:      return Continue(Post) ▷ Stay in Post
26:    end if
27:  end if
28: end function
```

- **PIGEND**: signals successful completion. The turn ends, and the accumulated visible text is returned to the client. **PIGEND**: 表示成功完成。本轮结束，累积的可见文本返回给客户端。
- **PIGFAIL**: signals that the current approach has failed and the task should be replanned from Pre. **PIGFAIL**: 表示当前方法已失败，任务应从 Pre 重新规划。

Marker detection follows two rules: 标记检测遵循两条规则：

Last non-empty line only. Only the final non-empty line of the model’s output is checked for a marker. Earlier occurrences (e.g., the marker appearing in a sentence like “the system outputs PIGEND when done”) are ignored. This prevents false positives from prose mentions. **仅检测最后一个非空行。** 仅检查模型输出的最后一个非空行是否包含标记。早期出现的标记（例如标记出现在“系统在完成时输出 PIGEND”这样的句子中）会被忽略。这防止了散文中提及标记导致的误报。

Reason required. A marker is valid only if at least one non-marker line precedes it. A bare PIGEND with no preceding content is rejected, forcing the model to provide a reason before signaling completion. **要求附带理由。** 标记仅在前面至少有一个非标记行时才有效。单独的 PIGEND 且无前置内容将被拒绝，从而强制模型在发出完成信号前提供理由。

Markers are stripped from the visible output via **strip_markers**, which removes exact control lines while preserving prose that happens to contain marker words. 标记通过 **strip_markers** 从可见输出中剥离，该函数移除精确的控制行，同时保留恰好包含标记词的散文。

3.4 Protocol-Native Request Envelope

The `HttpRequestEnvelope` is the central data structure that preserves the complete protocol-native request: `HttpRequestEnvelope` 是保留完整协议原生请求的核心数据结构：

```
pub struct HttpRequestEnvelope {
  pub method: String,      // HTTP method
  pub path_and_query: String, // path + query string
  pub headers: Vec<HeaderPair>, // raw header bytes
  pub body: Value,         // complete JSON body
```

```

pub protocol: Protocol,          // Chat / Anthropic / Responses
pub client_model: String,        // e.g. "gpt-4-pig"
pub real_model: String,          // e.g. "gpt-4" (-pig stripped)
pub stream: bool,                // streaming requested?
// ... internal fields for current user location,
//      tool result IDs, tool result groups
}

```

Each phase clones the envelope and applies *point mutations*: 每个阶段克隆该封装并施加定点修改:

1. **Model**: replaced with the real (non-`-pig`) model name. **模型**: 替换为真实 (非 `-pig`) 模型名。
2. **Current user input**: appended with the phase-specific prompt suffix (Pre questions, Executor plan, Post review instructions). **当前用户输入**: 附加阶段特定的提示后缀 (Pre 问题、Executor 计划、Post 审查指令)。
3. **Phase transcript**: native assistant/tool messages from prior phases are appended to the message history. **阶段记录**: 来自先前阶段的原生 assistant/tool 消息被附加到消息历史中。
4. **Stream**: set to `true` for streaming phase subrequests. **流式**: 对于流式阶段子请求设为 `true`。

All other fields—system/instructions, historical messages, tools/tool_choice, reasoning parameters, cache controls, metadata, media blocks, and unknown provider extensions—remain untouched. This ensures that provider-specific features (e.g., Anthropic’s `cache_control`, OpenAI’s `reasoning_effort`) pass through without modification. 所有其他字段——system/instructions、历史消息、tools/tool_choice、推理参数、缓存控制、元数据、媒体块以及未知的提供商扩展——均保持不变。这确保了提供商特定特性 (如 Anthropic 的 `cache_control`、OpenAI 的 `reasoning_effort`) 原样透传, 无需修改。

3.5 HTTP Loopback Transport

Each phase subrequest is sent back to the local proxy via HTTP, not through an in-process function call. The `LoopbackPhaseTransport` implements the `PhaseTransport` trait: 每个阶段子请求通过 HTTP 发送回本地代理, 而非通过进程内函数调用。`LoopbackPhaseTransport` 实现了 `PhaseTransport` trait:

- Converts wildcard listen addresses (`0.0.0.0, [::]`) to loopback (`127.0.0.1, ::1`). 将通配监听地址 (`0.0.0.0, [::]`) 转换为回环地址 (`127.0.0.1, ::1`)。
- Disables environment HTTP proxy settings (`.no_proxy()`) to prevent external proxy interception. 禁用环境 HTTP 代理设置 (`.no_proxy()`), 以防止外部代理拦截。
- Adds a random internal token header (`x-pigs-phase-loopback`) that the proxy validates to identify legitimate phase subrequests. 添加随机内部令牌头部 (`x-pigs-phase-loopback`), 由代理验证以识别合法的阶段子请求。
- Strips hop-by-hop headers and the internal token before forwarding to the upstream provider. 在转发至上游提供商前剥离逐跳头部和内部令牌。

This design has two benefits. First, phase subrequests transparently inherit all proxy-level features: channel selection, model mapping, body cleaning (e.g., removing empty content messages), thinking-effort injection, and retry logic. Second, the control flow is explicit and debuggable—each phase subrequest appears in proxy logs as a normal HTTP request, and the internal token distinguishes it from client requests. 此设计有两个优势。第一, 阶段子请求透明地继承所有代理级特性: 通道选择、模型映射、主体清洗 (如移除空内容消息)、思考力度注入和重试逻辑。第二, 控制流是显式且可调试的——每个阶段子请求在代理日志中作为普通 HTTP 请求出现, 内部令牌使其与客户端请求区分开来。

3.6 Bounded In-Memory Continuation

When the model returns tool calls during a phase, the runtime pauses and returns the tool calls to the client (or external agent). The complete phase state—including the orchestration state, phase transcript, visible text, and pending tool IDs—is stored in a `ContinuationStore`: 当模型在某个阶段返回工具调用时, 运行时暂停并将工具调用返回给客户端 (或外部智能体)。完整的阶段状态——包括编排状态、阶段记录、可见文本和待处理工具 ID——存储在 `ContinuationStore` 中:

- **TTL**: each continuation expires after a configurable duration (default 30 minutes). **TTL**: 每个续传在可配置的时长后过期 (默认 30 分钟)。
- **Capacity**: at most N continuations are retained (default 256); the oldest is evicted when capacity is reached. **容量**: 最多保留 N 个续传 (默认 256); 达到容量时驱逐最旧的续传。
- **Tombstones**: when a continuation is expired, evicted, or consumed, a tombstone is placed for each tool ID, enabling distinct error reporting (Expired vs. Evicted vs. Consumed vs. Unknown).

墓碑：当续传过期、被驱逐或被消费时，为每个工具 ID 放置一个墓碑，实现差异化错误报告（Expired 与 Evicted 与 Consumed 与 Unknown）。

- **No auth headers**: authentication headers are cleared before storing; the continuation uses the incoming request's headers on resume. **不存储认证头部**：认证头部在存储前被清除；续传在恢复时使用传入请求的头部。

When the next request arrives with tool results, the runtime matches them by tool-call ID, validates protocol and model consistency, and resumes the paused phase. Partial parallel results (multiple tool calls returning in separate requests) are accumulated until all pending IDs are present. 当下一个请求携带工具结果到达时，运行时通过工具调用 ID 匹配它们，验证协议和模型一致性，并恢复暂停的阶段。部分并行结果（多个工具调用在单独请求中返回）会被累积，直到所有待处理 ID 均到齐。

3.7 Continuous SSE Output

For streaming requests, the runtime produces a continuous SSE stream that concatenates visible text from all phases. A *marker line buffer* holds back the last non-empty line of each phase's output until the phase completes, then checks for control markers. If a marker is found, the held-back line is stripped; otherwise, it is forwarded. This prevents PIGEND/PIGFAIL from leaking into the client-visible stream, even when markers are split across network chunks. 对于流式请求，运行时产生连续的 SSE 流，拼接所有阶段的可见文本。标记行缓冲区暂缓每个阶段输出的最后一个非空行，直到该阶段完成，然后检查控制标记。若发现标记，被暂缓的行将被剥离；否则，将其转发。这防止了 PIGEND/PIGFAIL 泄漏到客户端可见的流中，即使标记跨越网络数据块的分界也是如此。

The outer SSE encoder generates protocol-native event sequences for all three protocols: 外层 SSE 编码器为所有三种协议生成协议原生的事件序列：

- **Chat**: chat.completion.chunk deltas with role/content/tool_calls, terminated by finish_reason + [DONE]. **Chat**: chat.completion.chunk 增量，附带 role/content/tool_calls，以 finish_reason + [DONE] 终止。
- **Anthropic**: message_start → content_block_start/delta/stop → message_delta → message_stop. **Anthropic**: message_start → content_block_start/delta/stop → message_delta → message_stop。
- **Responses**: response.created → output_item/content_part/text_delta/done → response.completed, with monotonic sequence numbers. **Responses**: response.created → output_item/content_part/text_delta/done → response.completed, 带有单调递增的序列号。

Error streams emit protocol-native error events (Chat: error in data; Anthropic: event: error; Responses: response.failed) without any success terminal frame. 错误流发出协议原生的错误事件（Chat: 数据中的 error; Anthropic: event: error; Responses: response.failed），不含任何成功的终止帧。

4 Implementation

Pigs is implemented as a Rust workspace with 14 crates, organized in five layers: Pigs 以 Rust workspace 形式实现，包含 14 个 crate，组织为五个层次：

Layer	Crates	Role
0	pigs-core	Core types: Message, ContentBlock, ApiClient trait
1	pigs-config, pigs-permissions, pigs-session, pigs-prompts	Config, permissions, session, prompts
2	pigs-llm, pigs-tools, pigs-mcp	LLM clients, built-in tools, MCP client
3	pigs-api, pigs-proxy	Phased runtime, HTTP proxy + loopback
4	pigs-cli, pigs-tui, pigs	Agent logic, terminal UI, product binary

4.1 Protocol Codec

The ProtocolCodec in pigs-api handles parsing, mutation, and response extraction for all three protocols. It validates the “current user input” (the last user message, not just the last non-system message) and identifies trailing tool-result groups for continuation matching. pigs-api 中的 ProtocolCodec 负责所有三种协议的解析、修改和响应提取。它验证“当前用户输入”（最后一条 user 消息，而非仅仅是最后一条非系统消息），并识别尾部的工具结果组以进行续传匹配。

Each protocol has dedicated parsing logic: 每种协议具有专用的解析逻辑：

- **Chat:** messages array with role-based content; tool calls as `tool_calls` array; tool results as `role: "tool"` messages. **Chat:** 具有基于角色内容的 `messages` 数组; 工具调用为 `tool_calls` 数组; 工具结果为 `role: "tool"` 消息。
- **Anthropic:** messages with content block arrays; tool calls as `tool_use` blocks; tool results as `tool_result` blocks within user messages. **Anthropic:** 具有内容块数组的 `messages`; 工具调用为 `tool_use` 块; 工具结果为 `user` 消息内的 `tool_result` 块。
- **Responses:** input array with typed items; tool calls as `function_call` items; tool results as `function_call_output` items. **Responses:** 具有类型化条目的 `input` 数组; 工具调用为 `function_call` 条目; 工具结果为 `function_call_output` 条目。

The codec clones the original body and modifies only the JSON path leading to the current user input, leaving all other fields structurally identical. 编解码器克隆原始 `body` 并仅修改指向当前用户输入的 JSON 路径, 保持所有其他字段在结构上完全一致。

4.2 HTTP Runtime

The `HttpPhasedRuntime` drives the phase loop. For each phase, it: `HttpPhasedRuntime` 驱动阶段循环。对于每个阶段, 它:

1. Constructs a phase request via `phase_request()` (Pre/Executor/Post prompt + transcript). 通过 `phase_request()` 构建阶段请求 (Pre/Executor/Post 提示 + 记录)。
2. Sends the request via `PhaseTransport::send_streaming()` (streaming) or `send()` (non-streaming). 通过 `PhaseTransport::send_streaming()` (流式) 或 `send()` (非流式) 发送请求。
3. Extracts the normalized model output (visible text, tool calls, usage, stop reason). 提取归一化的模型输出 (可见文本、工具调用、使用量、停止原因)。
4. Checks for tool calls → pauses and stores continuation. 检查工具调用 → 暂停并存储续传。
5. Advances the state machine via `OrchestrationState::advance()`. 通过 `OrchestrationState::advance()` 推进状态机。

The runtime is transport-agnostic: it depends only on the `PhaseTransport` trait, not on HTTP or axum. This allows the same runtime to be used with any transport implementation. 运行时与传输无关: 它仅依赖于 `PhaseTransport` trait, 不依赖 HTTP 或 axum。这使得同一运行时可与任何传输实现配合使用。

4.3 Loopback Transport

The `LoopbackPhaseTransport` in `pigs-proxy` sends phase subrequests via HTTP to the local proxy. It parses both non-streaming JSON and SSE responses from the upstream provider, aggregating: `pigs-proxy` 中的 `LoopbackPhaseTransport` 通过 HTTP 将阶段子请求发送至本地代理。它解析来自上游提供商的非流式 JSON 和 SSE 响应, 聚合:

- Chat: text deltas, tool call deltas (merged by index), finish reason, usage, provider extensions. **Chat:** 文本增量、工具调用增量 (按索引合并)、完成原因、使用量、提供商扩展。
- Anthropic: content blocks (text, thinking, signature, `tool_use` with `input_json_delta` accumulation), stop reason, usage. **Anthropic:** 内容块 (text、thinking、signature、`tool_use` 及 `input_json_delta` 累积)、停止原因、使用量。
- Responses: output items (message, reasoning, `function_call`), status, incomplete details, usage. **Responses:** 输出条目 (message、reasoning、`function_call`)、状态、不完整详情、使用量。

Malformed streams (missing termination events, unknown content block indices, open blocks at stream end) are reported as typed errors, not silently accepted. 格式错误的流 (缺少终止事件、未知内容块索引、流结束时未关闭的块) 被报告为类型化错误, 而非被静默接受。

4.4 Body Cleaning

The proxy's `clean_empty_messages` function removes messages with empty content, but preserves assistant messages that contain `tool_calls` (even if their content is empty). For messages with empty content and tool calls, the content is replaced with a single space to satisfy upstream providers that reject empty strings. 代理的 `clean_empty_messages` 函数移除内容为空的消息, 但保留包含 `tool_calls` 的 assistant 消息 (即使其内容为空)。对于内容为空且含工具调用的消息, 内容被替换为单个空格, 以满足上游提供商对空字符串的拒绝要求。

4.5 Agent Client and Terminal UI

Pigs ships with a general-purpose agent client (similar to PI [Team, 2026] or OpenCode), which connects to the local API server via HTTP. The key design point is that the client can use either model identifier: `{model}` (passthrough, no orchestration) or `{model}-pig` (phased orchestration). This allows the same agent to toggle orchestration on or off simply by changing the model name in the API request, with no other code changes. Pigs 附带一个通用 Agent 客户端（类似 PI [Team, 2026] 或 OpenCode），通过 HTTP 连接本地 API 服务器。关键设计在于：客户端可以使用任一模型标识符——`{model}`（透传，无编排）或 `{model}-pig`（相位编排）。这使得同一 Agent 仅通过更改 API 请求中的模型名即可开启或关闭编排，无需其他代码变更。

The user interface is a full-screen terminal UI built with **ratatui** and **crossterm**, implemented in the `pigs-tui` crate. The layout follows PI’s convention, with five vertically-stacked regions: (1) a header bar showing the version, model, and keybinding hints; (2) a scrollable chat history rendering user messages, assistant responses (with Markdown formatting via `pulldown-cmark`), tool call results, and bash output; (3) a status line with a working spinner; (4) a multi-line text editor (Emacs-style keybindings via `tui-textarea`); and (5) a footer status bar displaying the working directory, git branch, token statistics, and current model. 用户界面为全屏终端 UI，使用 **ratatui** 和 **crossterm** 实现，位于 `pigs-tui` crate 中。布局遵循 PI 的约定，包含五个垂直堆叠区域：(1) 顶部栏显示版本、模型和快捷键提示；(2) 可滚动的聊天历史区，渲染用户消息、助手回复（通过 `pulldown-cmark` 进行 Markdown 格式化）、工具调用结果和 bash 输出；(3) 状态行显示工作旋转指示器；(4) 多行文本编辑器（通过 `tui-textarea` 实现 Emacs 风格快捷键）；(5) 底部状态栏显示工作目录、git 分支、token 统计和当前模型。

Streaming output is handled through an event broker architecture: the agent’s `run_turn_with_callback` method forwards `TextDelta`, `ToolStart`, and `ToolEnd` events from the phased runtime’s `ProgressSink` to the TUI via a `tokio::mpsc` channel. This enables real-time, token-by-token rendering of the model’s response, live tool execution display, and phase-aware status updates. The TUI gracefully falls back to a line-based REPL (via `rustyline`) if terminal initialization fails. 流式输出通过事件代理架构处理：Agent 的 `run_turn_with_callback` 方法将相位运行时 `ProgressSink` 的 `TextDelta`、`ToolStart` 和 `ToolEnd` 事件通过 `tokio::mpsc` 通道转发到 TUI。这实现了模型响应的实时逐 token 渲染、工具执行的实时显示和相位感知状态更新。如果终端初始化失败，TUI 会优雅地回退到基于行的 REPL（通过 `rustyline`）。

The built-in tool set is aligned with PI: `read`, `bash`, `edit`, and `write` are enabled by default, with optional `grep`, `find`, and `ls`. Configuration paths follow PI’s convention: `~/.pig/` for global state (sessions, logs, skills, config) and `.pig/` for project-local overrides. 内置工具集与 PI 对齐：`read`、`bash`、`edit` 和 `write` 默认启用，可选启用 `grep`、`find` 和 `ls`。配置路径遵循 PI 的约定：`~/.pig/` 用于全局状态（会话、日志、技能、配置），`.pig/` 用于项目级覆盖。

5 Experiments

To evaluate whether phased orchestration improves task completion quality, we design four experiments that collectively address the concerns raised in prior review: (1) an ablation study isolating each phase’s contribution, (2) a control marker effectiveness study, (3) a three-protocol consistency evaluation, and (4) a cost-benefit analysis. All experiments are conducted through the Pigs HTTP API, requiring only API access—no GPU resources. 为评估分阶段编排是否能提升任务完成质量，我们设计了四组实验，以共同回应先前评审中提出的关注点：(1) 隔离各阶段贡献的消融研究，(2) 控制标记有效性研究，(3) 三协议一致性评估，(4) 成本效益分析。所有实验均通过 Pigs HTTP API 执行，仅需 API 访问——无需 GPU 资源。

5.1 Experimental Setup

System configuration. All experiments use the Pigs API server with default orchestration limits (`max_post_iterations=3`, `max_pre_replans=2`), continuation TTL of 30 minutes, capacity of 256, and Chinese (zh) phase prompts. Thinking effort is set to `xhigh` for OpenAI and `max` for Anthropic. **系统配置。**所有实验使用 Pigs API 服务器，采用默认编排限制 (`max_post_iterations=3`, `max_pre_replans=2`)，续传 TTL 为 30 分钟，容量为 256，阶段提示采用中文 (zh)。思考力度对 OpenAI 设为 `xhigh`，对 Anthropic 设为 `max`。

Benchmarks. We use three public benchmarks: **基准测试**。我们使用三个公开基准测试：

Benchmark	Domain	Tasks	Public
SWE-bench Lite [Jimenez et al., 2024]	Software engineering	300	✓
GAIA [Mialon et al., 2023]	General AI assistant	300	✓
AgentBoard [Ma et al., 2024]	Multi-turn agent	9 tasks	✓

Ablation conditions. For Experiment 1, we define four conditions: **消融条件**。对于实验一，我们定义四种条件：

- **Condition A (Baseline):** Direct API call without orchestration (non-`-pig` model, passthrough). **条件 A (基线):** 不经过编排的直接 API 调用 (非 `-pig` 模型, 透传)。
- **Condition B (Full):** Complete Pre→Executor→Post orchestration (`-pig` model). **条件 B (完整):** 完整的 Pre→Executor→Post 编排 (`-pig` 模型)。
- **Condition C (Pre-only):** Only the Pre phase (planning without execution verification). **条件 C (仅 Pre):** 仅 Pre 阶段 (规划但不执行验证)。
- **Condition D (Executor+Post):** Skip Pre, directly execute and review. **条件 D (Executor+Post):** 跳过 Pre, 直接执行并审查。

Metrics. We measure: (1) task completion rate (benchmark-specific), (2) average LLM API calls per task, (3) total input + output tokens per task, (4) wall-clock latency, and (5) error recovery rate (percentage of tasks where PIGFAIL triggered a successful replan). **指标。**我们测量：(1) 任务完成率 (基准测试特定), (2) 每任务平均 LLM API 调用次数, (3) 每任务总输入 + 输出令牌数, (4) 端到端延迟, (5) 错误恢复率 (PIGFAIL 触发成功重新规划的任务百分比)。

5.2 Experiment 1: Ablation Study

Setup. We evaluate four conditions (A: baseline, B: full orchestration, C: Pre-only, D: Executor+Post) on SWE-bench Lite (300 tasks) and GAIA (300 tasks). Each task is sent as a `-pig` model request (Condition B/C/D) or a passthrough request (Condition A) to the Pigs API. **设置。**我们在 SWE-bench Lite (300 个任务) 和 GAIA (300 个任务) 上评估四种条件 (A: 基线, B: 完整编排, C: 仅 Pre, D: Executor+Post)。每个任务作为 `-pig` 模型请求 (条件 B/C/D) 或透传请求 (条件 A) 发送至 Pigs API。

Procedure. For each task: **流程。**对于每个任务：

1. Send the task prompt to the Pigs API endpoint. 将任务提示发送至 Pigs API 端点。
2. If tool calls are returned, execute them locally and send results back. 若返回工具调用, 在本地执行并将结果发回。
3. Repeat until the API returns a final response (no tool calls). 重复直至 API 返回最终响应 (无工具调用)。
4. Evaluate the final response against the benchmark’s ground truth. 将最终响应与基准测试的标准答案进行评估。

Hypothesis. Full orchestration (B) outperforms baseline (A) on task completion rate, with the Pre phase contributing planning and the Post phase contributing verification. **假设。**完整编排 (B) 在任务完成率上优于基线 (A), 其中 Pre 阶段贡献规划能力, Post 阶段贡献验证能力。

5.3 Experiment 2: Control Marker Effectiveness

Setup. We compare two configurations on GAIA: (A) with PIGFAIL enabled (failure triggers replan), (B) with PIGFAIL disabled (failure treated as completion). **设置。**我们在 GAIA 上比较两种配置: (A) 启用 PIGFAIL (失败触发重新规划), (B) 禁用 PIGFAIL (失败被视为完成)。

Procedure. We count: (1) tasks where PIGFAIL was emitted, (2) tasks where replan led to successful completion, (3) tasks that failed despite replan. **流程。**我们统计: (1) 发出 PIGFAIL 的任务数, (2) 重新规划后成功完成的任务数, (3) 尽管重新规划仍然失败的任务数。

Hypothesis. PIGFAIL-based replan improves outcomes on tasks where the initial approach was flawed. **假设。**基于 PIGFAIL 的重新规划在初始方法有缺陷的任务上能改善结果。

5.4 Experiment 3: Three-Protocol Consistency

Setup. We send the same set of 100 tasks through three API protocols: OpenAI Chat (`/chat/completions`), Anthropic Messages (`/v1/messages`), and OpenAI Responses (`/responses`). **设置。**我们将同一组 100 个任务通过三种 API 协议发送: OpenAI Chat (`/chat/completions`)、Anthropic Messages (`/v1/messages`) 和 OpenAI Responses (`/responses`)。

Procedure. For each task and protocol, we record: (1) whether the task completed, (2) the visible text output, (3) any tool calls made. We measure consistency as the percentage of tasks where all three protocols agree on completion status. **流程.** 对于每个任务和协议，我们记录：（1）任务是否完成，（2）可见文本输出，（3）所执行的工具调用。我们将一致性定义为三种协议在完成状态上达成一致的任务百分比。

Hypothesis. The protocol-native request preservation ensures consistent behavior across all three protocols. **假设.** 协议原生请求保留确保了三种协议间行为的一致性。

5.5 Experiment 4: Cost-Benefit Analysis

Setup. On SWE-bench Lite, we measure per-task: (1) total input tokens, (2) total output tokens, (3) number of API calls, (4) wall-clock latency, (5) task completion. **设置.** 在 SWE-bench Lite 上，我们逐任务测量：（1）总输入令牌数，（2）总输出令牌数，（3）API 调用次数，（4）端到端延迟，（5）任务完成情况。

Procedure. We compare Condition A (baseline) vs. Condition B (full orchestration) and compute the overhead ratio for each metric. **流程.** 我们比较条件 A（基线）与条件 B（完整编排），并计算各指标的开销比率。

Hypothesis. The token overhead of orchestration (3 phases vs. 1) is justified by improved completion rate. **假设.** 编排的令牌开销（3 阶段 vs. 1 阶段）可通过提升的完成率来证明其合理性。

6 Results

[All experimental results are pending. The tables below will be filled upon completion of the evaluation.] [所有实验结果待补充。下述表格将在评估完成后填写。]

6.1 Experiment 1: Ablation Results

Benchmark	Condition	Completion Rate (%)	Avg. Turns	Tokens (k)	Latency (s)
SWE-bench Lite	A (Baseline)	—	—	—	—
	B (Full)	—	—	—	—
	C (Pre-only)	—	—	—	—
	D (Exec+Post)	—	—	—	—
GAIA	A (Baseline)	—	—	—	—
	B (Full)	—	—	—	—
	C (Pre-only)	—	—	—	—
	D (Exec+Post)	—	—	—	—

6.2 Experiment 2: Control Marker Results

Configuration	PIGFAIL emitted (%)	Replan succeeded (%)	Overall completion (%)
A (With PIGFAIL)	—	—	—
B (Without PIGFAIL)	N/A	N/A	—

6.3 Experiment 3: Protocol Consistency

Protocol	Completion (%)	Tool calls	Text agreement	Consistency
OpenAI Chat	—	—	—	—
Anthropic Messages	—	—	—	—
OpenAI Responses	—	—	—	—

6.4 Experiment 4: Cost-Benefit

Condition	Input tokens (k)	Output tokens (k)	API calls	Completion (%)
A (Baseline)	—	—	—	—
B (Full)	—	—	—	—
Overhead	—×	—×	—×	—pp

7 Related Work

7.1 Multi-Stage LLM Reasoning

ReAct [Yao et al., 2023b] pioneered interleaved reasoning and acting in LLMs. Unlike Pigs, ReAct embeds control flow inside the model’s output—the model decides when to reason and when to act. Pigs externalizes control flow to a deterministic state machine, using the model only for semantic content and routing signals. **ReAct** [Yao et al., 2023b] 开创了 LLM 中推理与行动交替的范式。与 Pigs 不同, ReAct 将控制流嵌入模型输出中——由模型决定何时推理、何时行动。Pigs 将控制流外置至确定性状态机, 模型仅用于语义内容和路由信号。

Reflexion [Shinn et al., 2023] adds verbal reinforcement learning through reflective feedback. The reflection trigger is model-driven, whereas Pigs’ Post phase is a mandatory structural step enforced by the state machine. **Reflexion** [Shinn et al., 2023] 通过反思反馈增加了语言强化学习。其反思触发是模型驱动的, 而 Pigs 的 Post 阶段是由状态机强制的必选结构性步骤。

Self-Refine [Madaan et al., 2023] uses a single LLM as generator, critic, and refiner. Pigs separates these roles into distinct phases with distinct prompts, and adds a Pre (planning) phase that Self-Refine lacks. **Self-Refine** [Madaan et al., 2023] 使用单个 LLM 充当生成器、批评器和精炼器。Pigs 将这些角色分离为具有不同提示的不同阶段, 并增加了 Self-Refine 所缺乏的 Pre (规划) 阶段。

Plan-and-Solve [Wang et al., 2023b] separates planning from execution in prompt structure. Pigs extends this to a three-phase model with an explicit review phase and failure-recovery routing. **Plan-and-Solve** [Wang et al., 2023b] 在提示结构中将规划与执行分离。Pigs 将其扩展为具有显式审查阶段和故障恢复路由的三阶段模型。

Tree of Thoughts [Yao et al., 2023a] and **Graph of Thoughts** [Besta et al., 2024] generalize chain-of-thought to search structures. These approaches require the model to self-evaluate states, while Pigs uses a dedicated Post phase for evaluation. **Tree of Thoughts** [Yao et al., 2023a] 和 **Graph of Thoughts** [Besta et al., 2024] 将思维链推广为搜索结构。这些方法要求模型自我评估状态, 而 Pigs 使用专门的 Post 阶段进行评估。

7.2 LLM-as-Code and Deterministic Control

Recent work has proposed shifting control flow from the LLM to deterministic program structures. **LLM-as-Code** [others, 2026b] treats the LLM as an adaptive component that cannot alter the program’s execution path. **Compile, Then Page** [others, 2026a] compiles standard operating procedures into executable pseudo-code with a stack machine. **XFlow** [others, 2026e] defines a DSL with lifecycle-governed symbols that stage uncertainty. These systems share Pigs’ philosophy of deterministic control flow, but operate at a higher abstraction level (workflow DSLs, compiled programs). Pigs achieves deterministic control at the API middleware level, requiring no DSL or compilation step. 近期工作提出将控制流从 LLM 转移至确定性程序结构。**LLM-as-Code** [others, 2026b] 将 LLM 视为无法改变程序执行路径的自适应组件。**Compile, Then Page** [others, 2026a] 将标准操作流程编译为带有栈机的可执行伪代码。**XFlow** [others, 2026e] 定义了具有生命周期治理符号的 DSL 来分阶段处理不确定性。这些系统与 Pigs 共享确定性控制流的理念, 但运行在更高的抽象层次 (工作流 DSL、编译程序)。Pigs 在 API 中间件层面实现了确定性控制, 无需 DSL 或编译步骤。

Rel(AI)Build [others, 2026c] proposes a deterministic control plane for coding agents with a phase state machine. This is architecturally similar to Pigs, but focuses on supply-chain integrity (SHA-256 addressing, HMAC lockfiles) rather than API-level orchestration. **Rel(AI)Build** [others, 2026c] 提出了具有阶段状态机的编码智能体确定性控制平面。这在架构上与 Pigs 相似, 但专注于供应链完整性 (SHA-256 寻址、HMAC 锁定文件), 而非 API 级编排。

7.3 LLM Middleware and Proxies

A position paper [Dzanic et al., 2024] proposes middleware for enterprise LLM deployment, but provides no implementation. Pigs is an implemented system that realizes this vision with a specific orchestration model. **Towards a Middleware for LLMs** [Dzanic et al., 2024] 是一篇愿景论文，提出了面向企业 LLM 部署的中间件，但未提供实现。Pigs 是一个已实现的系统，以特定的编排模型实现了这一愿景。

TokenMizer [Liu, 2026] adds session state management to the proxy layer. While it introduces state at the proxy level, it does not impose phase structure or control markers. **TokenMizer** [Liu, 2026] 是一个透明代理，通过类型化知识图谱管理会话状态。虽然它在代理层增加了状态，但不施加阶段结构或控制标记。

STREAM [others, 2026d] is a multi-tier LLM inference routing system (local→HPC→cloud) with a complexity judge. Pigs’ routing is phase-based (Pre→Executor→Post), not tier-based. **STREAM** [others, 2026d] 实现了具有复杂度判定器的多级 LLM 推理路由（本地 →HPC→ 云）。Pigs 的路由是基于阶段的（Pre→Executor→Post），而非基于层级。

Dyserve [others, 2027] is a workflow-aware serving layer that reroutes on tool-call failures. Pigs’ **PIGFAIL** marker provides a similar failure-recovery mechanism, but at the phase level rather than the tool-call level. **Dyserve** [others, 2027] 是一个工作流感知的服务层，在工具调用失败时重新路由。Pigs 的 **PIGFAIL** 标记提供了类似的故障恢复机制，但在阶段层面而非工具调用层面。

7.4 Tool-Use Agent Frameworks

ToolLLM/ToolBench [Qin et al., 2024] provides a framework and benchmark for tool-augmented LLMs. **MetaGPT** [Hong et al., 2024] encodes SOPs into prompt sequences. **MemGPT** [Packer et al., 2023] manages control flow via OS-inspired interrupts. Pigs differs by keeping tool execution external to the orchestration layer—the runtime pauses and returns tool calls to the client, rather than executing them internally. **ToolLLM/ToolBench** [Qin et al., 2024] 提供了工具增强 LLM 的框架和基准测试。**MetaGPT** [Hong et al., 2024] 将标准操作流程编码为提示序列。**MemGPT** [Packer et al., 2023] 通过操作系统启发的中断机制管理控制流。Pigs 的不同之处在于将工具执行保持在编排层之外——运行时暂停并将工具调用返回给客户端，而非在内部执行。

7.5 Multi-Model Orchestration

Sakana Fugu [AI, 2026] is a multi-agent system that dynamically orchestrates a pool of frontier models to tackle complex, multi-step tasks, delivering the multi-agent system as a single model API. Fugu uses a Coordinator/Conductor architecture to route sub-tasks to specialized models, supporting both Chat Completions and Responses endpoints. Pigs shares Fugu’s philosophy of transparent orchestration at the API level, but differs fundamentally: Fugu orchestrates *across* models (selecting different models for different sub-tasks), while Pigs orchestrates *within* a single model’s API call sequence (enforcing a Pre→Executor→Post phase structure). The two approaches are complementary—Pigs’ phase structure could be applied to each model in a Fugu-style pool. **Sakana Fugu** [AI, 2026] 是一个多智能体系统，通过动态编排前沿模型池来处理复杂的多步骤任务，以单一模型 API 的形式交付多智能体系统。Fugu 使用 Coordinator/Conductor 架构将子任务路由到专用模型，同时支持 Chat Completions 和 Responses 端点。Pigs 与 Fugu 共享在 API 层面透明编排的理念，但存在根本性差异：Fugu 编排的是跨模型（为不同子任务选择不同模型），而 Pigs 编排的是单一模型 API 调用序列内部的结构（强制执行 Pre→Executor→Post 相位结构）。两种方法是互补的——Pigs 的相位结构可以应用于 Fugu 式模型池中的每个模型。

Oh-My-OpenAgent (OmO) [Team, 2026] is a plugin-based agent operating system featuring “Team Mode,” where a lead agent coordinates up to 8 parallel members with real-time tmux visualization. OmO’s Team Mode implements multi-agent coordination with dedicated **team_*** tools, and powers features like **hyperplan** (5 hostile critics) and **security-research** (3 hunters + 2 PoC engineers). Pigs’ **PIGFAIL** marker serves a similar role to OmO’s hostile-critic pattern—both detect when an approach has failed and trigger replanning—but Pigs operates at the API middleware level with a single model, while OmO operates at the multi-agent coordination level with separate model instances. **AutoGen** [Wu et al., 2023] enables multi-agent conversation for complex task decomposition. Unlike these systems, Pigs does not require multiple model instances—its orchestration is intra-model, not inter-model. **Oh-My-OpenAgent (OmO)** [Team, 2026] 是一个基于插件的智能体操作系统，具有“Team Mode”功能，由主智能体协调最多 8 个并行成员，配有实时 tmux 可视化。OmO 的 Team Mode 通过专用 **team_*** 工具实现多智能体协调，并驱动 **hyperplan**（5 个对抗性评论者）和 **security-research**（3 个猎手 + 2 个 PoC 工程师）等功能。Pigs 的 **PIGFAIL** 标记与 OmO 的对抗性评论者模式扮演类似角色——两者都检测方法失败并触发重规划——但 Pigs 在 API 中间件层面操作单一模型，而 OmO 在

多智能体协调层面操作独立的模型实例。**AutoGen** [Wu et al., 2023] 通过多智能体对话实现复杂任务分解。与这些系统不同, Pigs 不需要多个模型实例——其编排是模型内部的, 而非模型间的。

7.6 Agent Benchmarks

Several benchmarks evaluate LLM agents: **AgentBench** [Liu et al., 2024a] (8 environments), **SWE-bench** [Jimenez et al., 2024] (software engineering), **GAIA** [Mialon et al., 2023] (general AI assistants), **ToolBench** [Qin et al., 2024] (tool use), and **AgentBoard** [Ma et al., 2024] (multi-turn analysis). These benchmarks evaluate end-to-end agent performance but do not isolate the contribution of orchestration structure. Pigs’ phased design enables ablation studies (e.g., Pre-only vs. full pipeline) that these benchmarks can measure. 若干基准测试用于评估 LLM 智能体: **AgentBench** [Liu et al., 2024a] (8 个环境)、**SWE-bench** [Jimenez et al., 2024] (软件工程)、**GAIA** [Mialon et al., 2023] (通用 AI 助手)、**ToolBench** [Qin et al., 2024] (工具使用) 和 **AgentBoard** [Ma et al., 2024] (多轮分析)。这些基准测试评估端到端智能体性能, 但不隔离编排结构的贡献。Pigs 的分阶段设计使得消融实验成为可能 (如仅 Pre 与完整流水线的对比), 这些基准测试可以测量。

7.7 Surveys and Foundations

Several surveys provide comprehensive background for this work. Xi et al. [2023] and Wang et al. [2023a] survey LLM-based autonomous agents, covering single-agent and multi-agent scenarios. Zhao et al. [2023] provides a broad survey of large language models. Qin et al. [2023] surveys tool learning with foundation models, defining a four-stage taxonomy (task planning, tool selection, tool calling, response generation) that maps to Pigs’ Pre→Executor→Post phases. Sumers et al. [2023] proposes cognitive architectures for language agents (CoALA), formalizing the distinction between reasoning, acting, and memory. Schulhoff et al. [2024] surveys prompt engineering techniques systematically. Kaddour et al. [2023] discusses challenges and applications of LLMs. 若干综述为本文提供了全面的背景。Xi et al. [2023] 和 Wang et al. [2023a] 综述了基于 LLM 的自主智能体, 涵盖单智能体和多智能体场景。Zhao et al. [2023] 提供了大语言模型的广泛综述。Qin et al. [2023] 综述了基础模型的工具学习, 定义了四阶段分类法 (任务规划、工具选择、工具调用、响应生成), 与 Pigs 的 Pre→Executor→Post 阶段相对应。Sumers et al. [2023] 提出了语言智能体的认知架构 (CoALA), 形式化了推理、行动和记忆之间的区分。Schulhoff et al. [2024] 系统地综述了提示工程技巧。Kaddour et al. [2023] 讨论了 LLM 的挑战与应用。

7.8 Self-Correction and Verification

The Post phase in Pigs is related to LLM self-correction. Huang et al. [2024] shows that LLMs cannot reliably self-correct reasoning without external feedback, motivating Pigs’ use of a dedicated review phase with structured prompts rather than relying on the model’s intrinsic self-correction. Asai et al. [2023] proposes Self-RAG for retrieval-augmented self-reflection. Lightman et al. [2023] demonstrates that step-by-step verification improves reasoning. Bai et al. [2022] introduces Constitutional AI for harmlessness through AI feedback. Peng et al. [2023] improves LLMs with external knowledge and automated feedback. Pigs 的 Post 阶段与 LLM 自我纠正相关。Huang et al. [2024] 表明 LLM 在没有外部反馈的情况下无法可靠地自我纠正推理, 这促使 Pigs 使用具有结构化提示的专门审查阶段, 而非依赖模型的内在自我纠正能力。Asai et al. [2023] 提出了 Self-RAG, 用于检索增强的自我反思。Lightman et al. [2023] 证明了逐步验证可以改善推理。Bai et al. [2022] 引入了 Constitutional AI, 通过 AI 反馈实现无害性。Peng et al. [2023] 通过外部知识和自动化反馈改进 LLM。

7.9 LLM Serving Systems

On the infrastructure side, **vLLM** [Kwon et al., 2023] introduces PagedAttention for efficient memory management in LLM serving. **Orca** [Yu et al., 2022] proposes distributed serving with iteration-level scheduling. **S-LoRA** [Sheng et al., 2023] serves thousands of concurrent LoRA adapters. These systems optimize *inference* performance, while Pigs optimizes *orchestration* structure—the two concerns are orthogonal and complementary. 在基础设施方面, **vLLM** [Kwon et al., 2023] 引入了 PagedAttention 以实现 LLM 服务中的高效内存管理。**Orca** [Yu et al., 2022] 提出了具有迭代级调度的分布式服务。**S-LoRA** [Sheng et al., 2023] 服务数千个并发 LoRA 适配器。这些系统优化推理性能, 而 Pigs 优化编排结构——两者正交且互补。

7.10 Security Considerations

Prompt injection attacks [Greshake et al., 2023, Liu et al., 2024b] are a concern for LLM middleware. Pigs’ architecture mitigates some risks by separating the orchestration layer from the model’s output: control markers are detected by the middleware, not by the model’s own reasoning, making it harder for injected content to hijack the control flow. However, a comprehensive security analysis is left for future work. 提示注入攻击 [Greshake et al., 2023, Liu et al., 2024b] 是 LLM 中间件的一个关注点。Pigs 的架构通过将编排层与模型输出分离来降低部分风险：控制标记由中间件检测，而非由模型自身的推理检测，这使得注入内容更难劫持控制流。然而，全面的安全分析留待未来工作。

7.11 Classical Foundations

The concept of using a state machine for control flow has deep roots in software engineering. Harel [1987] introduced statecharts as a visual formalism for complex systems. Pigs’ Pre→Executor→Post state machine with marker-based transitions can be seen as a lightweight instance of this classical pattern, applied at the LLM API middleware level. 使用状态机进行控制流的概念在软件工程中有着深厚的根基。Harel [1987] 引入了状态图作为复杂系统的可视化形式化方法。Pigs 的基于标记转移的 Pre→Executor→Post 状态机可被视为这一经典模式的轻量级实例，应用于 LLM API 中间件层面。

8 Discussion

8.1 Why Text Markers Instead of Tool-Based Handoff?

A natural alternative to text control markers is to inject an orchestration-specific tool (e.g., `finish_task`, `replan_task`) into the model’s tool set. We reject this approach for three reasons. First, it pollutes the tool namespace—the model must distinguish between user-defined tools (`bash`, `read_file`) and orchestration tools, creating confusion. Second, tool calls consume a round-trip: the model emits a tool call, the system executes it (trivially), and the model receives the result. Text markers are zero-overhead. Third, tool-based handoff couples the orchestration logic to the tool-calling protocol, making it protocol-specific. Text markers work universally across all protocols and even with models that do not support tool calling. 文本控制标记的一个自然替代方案是将编排特定的工具（如 `finish_task`、`replan_task`）注入模型的工具体系。我们拒绝这一方案，原因有三。第一，它污染了工具命名空间——模型必须区分用户定义的工具（`bash`、`read_file`）和编排工具，造成混淆。第二，工具调用消耗一次往返：模型发出工具调用，系统（简单地）执行它，模型接收结果。文本标记是零开销的。第三，基于工具的交接将编排逻辑与工具调用协议耦合，使其成为协议特定的。文本标记在所有协议中通用，甚至适用于不支持工具调用的模型。

8.2 Why Post Stays in Post (Not Back to Executor)?

When the Post phase produces output without a marker (indicating progress but not completion), the state machine continues in Post rather than returning to Executor. This design prevents infinite Executor↔Post loops: if the model oscillates between “do” and “review” without converging, the Post iteration budget will eventually exhaust and return an error. Returning to Executor would reset the loop without progress, creating a potential infinite cycle. 当 Post 阶段产生不含标记的输出（表示有进展但未完成）时，状态机继续停留在 Post 阶段，而非返回 Executor。此设计防止了 Executor↔Post 的无限循环：如果模型在“执行”和“审查”之间振荡而无法收敛，Post 迭代预算最终将耗尽并返回错误。返回 Executor 将在无进展的情况下重置循环，造成潜在的无限循环。

8.3 Why Phase Instructions Only in User Payload?

Phase-specific instructions (Pre questions, Executor plan, Post review criteria) are appended to the current user message, not injected as system prompts. This preserves the client’s original system prompt verbatim, which is important for two reasons. First, many providers cache system prompts for reduced latency and cost [Anthropic, 2024]; modifying the system prompt would invalidate the cache. Second, the system prompt often contains project-specific instructions (coding standards, safety rules) that should not be overridden by orchestration logic. 阶段特定指令（Pre 问题、Executor 计划、Post 审查标准）被附加到当前用户消息中，而非作为系统提示注入。这逐字保留了客户端的原始系统提示，其重要性有两个原因。第一，许多提供商缓存系统提示以降低延迟和成本 [Anthropic, 2024]；修改系统提示将使缓存失效。第二，系统提示通常包含项目特定指令（编码标准、安全规则），不应被编排逻辑覆盖。

8.4 Why HTTP Loopback Instead of In-Process Dispatch?

Phase subrequests could be dispatched in-process (bypassing HTTP), as was done in earlier versions of Pigs. We switched to HTTP loopback for two reasons. First, it ensures that phase subrequests transparently inherit all proxy-level features (channel selection, model mapping, body cleaning, thinking-effort injection, retry). With in-process dispatch, these features must be manually invoked, leading to code duplication and divergence. Second, HTTP loopback makes the control flow explicit and debuggable—each phase subrequest appears in proxy logs as a normal HTTP request, and the internal token distinguishes it from client requests. 阶段子请求可以在进程内分发（绕过 HTTP），如 Pigs 的早期版本所做。我们切换到 HTTP 回环有两个原因。第一，它确保阶段子请求透明地继承所有代理级特性（通道选择、模型映射、主体清洗、思考力度注入、重试）。在进程内分发时，这些特性必须被手动调用，导致代码重复和分歧。第二，HTTP 回环使控制流显式且可调试——每个阶段子请求在代理日志中作为普通 HTTP 请求出现，内部令牌使其与客户端请求区分开来。

8.5 Limitations

Marker robustness. The control-marker approach depends on the model’s ability to follow instructions about when to output PIGEND/PIGFAIL. In practice, models sometimes output markers in inappropriate contexts (e.g., quoting the marker in prose). The marker detection algorithm mitigates this by checking only the last non-empty line and requiring a preceding reason, but it is not foolproof. Empirical measurement of false positive and false negative rates is left for future evaluation. **标记鲁棒性。**控制标记方法依赖于模型遵循关于何时输出 PIGEND/PIGFAIL 的指令的能力。在实践中，模型有时会在不恰当的上下文中输出标记（如在散文中引用标记）。标记检测算法通过仅检查最后一个非空行并要求前置理由来缓解此问题，但并非万无一失。

Single-model assumption. The current design uses the same model for all three phases. Multi-model orchestration (e.g., a cheaper model for Pre, a stronger model for Executor) is architecturally possible but not yet implemented. **单模型假设。**当前设计在所有三个阶段使用同一模型。多模型编排（如在 Pre 阶段使用较便宜的模型，在 Executor 阶段使用更强的模型）在架构上是可行的，但尚未实现。

No formal verification. The state machine is tested with unit tests covering all transitions, but lacks formal verification (e.g., model checking) of properties like “the system always terminates” or “no phase is skipped.” **无形式化验证。**状态机通过覆盖所有转移的单元测试进行了测试，但缺乏形式化验证（如模型检测）来保证“系统总是终止”或“无阶段被跳过”等性质。

Protocol coverage. While three major protocols are supported, the system does not cover all provider-specific extensions (e.g., Google Gemini’s API). **协议覆盖。**虽然支持三种主要协议，但系统未覆盖所有提供商特定扩展（如 Google Gemini 的 API）。

9 Conclusion

We presented Pigs, a phased orchestration middleware for LLM API requests that combines a deterministic Pre→Executor→Post state machine with protocol-native request preservation, HTTP loopback transport, and bounded in-memory continuation. By elevating orchestration from a prompting technique to a middleware concern, Pigs separates deterministic control flow from stochastic LLM calls, making the orchestration transparent, debuggable, and protocol-agnostic. The system is implemented as a 13-crate Rust workspace and supports three major API protocols with real SSE streaming. 我们提出了 Pigs，一种面向 LLM API 请求的分阶段编排中间件，将确定性的 Pre→Executor→Post 状态机与协议原生请求保留、HTTP 回环传输和有界内存续传相结合。通过将编排从提示技术提升为中间件关注点，Pigs 将确定性控制流与随机性 LLM 调用分离，使编排变得透明、可调试且协议无关。该系统以 13 个 crate 的 Rust workspace 实现，支持三种主要 API 协议及真实 SSE 流式传输。

We designed an experimental evaluation framework comprising four experiments: an ablation study isolating each phase’s contribution, a control marker effectiveness study evaluating PIGFAIL-based failure recovery, a three-protocol consistency evaluation, and a cost-benefit analysis. All experiments use public benchmarks (SWE-bench, GAIA, AgentBoard) and require only API access—no GPU resources. **Experimental results are pending and will be reported upon completion of the evaluation.** 我们设计了一套包含四组实验的评估框架：隔离各阶段贡献的消融研究、评估基于 PIGFAIL 的故障恢复的控制标记有效性研究、三协议一致性评估，以及成本效益分析。所有实验均使用公开基准测试（SWE-bench、GAIA、AgentBoard），仅需 API 访问——无需 GPU 资源。**实验结果待补充，将在评估完成后报告。**

Future work includes: (1) multi-model phase orchestration (different models for different phases), (2) formal verification of the state machine, (3) extended benchmarks (ToolBench, BFCL), (4) security analysis against prompt injection attacks [Greshake et al., 2023], and (5) marker robustness improvements using fine-tuned detection models. 未来工作包括: (1) 多模型阶段编排 (不同阶段使用不同模型), (2) 状态机的形式化验证, (3) 扩展基准测试 (ToolBench、BFCL), (4) 针对提示注入攻击的安全分析 [Greshake et al., 2023], (5) 使用微调检测模型提升标记鲁棒性。

Appendix: Phase Prompt Templates

This appendix presents the phase prompt templates used by the Pigs API orchestration runtime. Each phase has a user-level instruction that is appended to the current user message at runtime. Both Chinese (zh) and English (en) versions are provided. The templates use `{pre_output}` and `{failure_paths}` as placeholder variables filled at runtime via string replacement.

A.1 Pre Phase (Planning) — User Payload

Chinese (zh)

以上是本任务要求
`{failure_paths}`
本次回答不需要具体执行任务，而是对任务进行执行前分析，
严格按照要求输出这5个问题的回答
1) 本任务需要哪些项目内部信息?
2) 本任务需要哪些项目外部信息?
3) 为本任务的执行制定执行计划
4) 总结本任务的目标 (`goal`)，用于判断任务是否完成
5) 判断本任务是否为简单问题。当且仅当1)~4) 均不需要或
极其简单时，才判定为简单问题，并必须给出理由，然后在
回答最后单独一行输出 `PIGEND`。若不是简单问题，不要输出 `PIGEND`

English (en)

The text above is the task requirement.
`{failure_paths}`
Do not execute the task in this response. Analyze it before
execution and answer exactly these five questions:
1. What project-internal information is needed?
2. What external information is needed?
3. What is the execution plan for the task?
4. What is the task goal used to decide whether it is complete?
5. Decide whether this is a simple task. Only when questions 1-4
need nothing or are extremely simple, classify it as simple,
give the reason, and output `PIGEND` alone on the last line.
If not simple, do not output `PIGEND`.

A.2 Executor Phase — User Payload

Chinese (zh)

以下是对本次任务在 1. 内部信息的需求 2. 外部信息的需求
3. 执行计划 4. 任务目标 5. 任务难度方面的分析:
`{pre_output}`
完成对内部信息和外部信息的获取后，按照执行计划全力完成任务目标

English (en)

The following is the complete analysis of 1. internal information,
2. external information, 3. execution plan, 4. task goal, and
5. task difficulty:
`{pre_output}`
After obtaining the required internal and external information,
follow the execution plan and fully complete the task goal.

A.3 Post Phase (Review) — User Payload

Chinese (zh)

根据设定的目标，独立验收结果，对任务的执行情况进行全面审查。
先输出评审报告，进行如下判断：
1. 判定目标是否完全完成。仅当任务完成时，在回答最后输出 `PIGEND`
2. 判定任务是否是执行错误的，也就是当前离目标的距离比执行前
更远，在回答最后输出 `PIGFAIL`

3. 如果是推进了任务，但没有完全完成任务：在输出评审报告后，继续执行任务，最后不输出标记

English (en)

Using the stated goal, independently verify the result and comprehensively review execution of the task. First output a review report, then make these judgments:

1. Decide whether the goal is fully complete. Only when it is complete, end the response with PIGEND.
2. Decide whether execution was wrong, meaning it moved farther from the goal than before execution. End with PIGFAIL.
3. If the task progressed but is not fully complete, `continue` executing it after the review report and do not output a marker.

References

- Sakana AI. Sakana fugu: Multi-model orchestration delivered as a single model api. *arXiv preprint arXiv:2606.21228*, 2026. Technical report.
- Anthropic. Prompt caching with claude. <https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching>, 2024.
- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection. *arXiv preprint arXiv:2310.11511*, 2023.
- Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawk, Lukas Gianinazzi, Joanna Gajda, Torsten Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefer. Graph of thoughts: Solving elaborate problems with large language models. In *AAAI Conference on Artificial Intelligence*, 2024.
- Tario Dzanic et al. Towards a middleware for large language models. *arXiv preprint arXiv:2411.14513*, 2024.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.
- David Harel. Statecharts: A visual formalism for complex systems. *Science of Computer Programming*, 8(3):231–274, 1987.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.
- Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- Jean Kaddour, Joshua Harris, Maximilian Mozes, Herbie Bradley, Roberta Raileanu, and Robert McHardy. Challenges and applications of large language models. *arXiv preprint arXiv:2307.10169*, 2023.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- others Liu. Tokenmizer: Graph-structured session memory for llm api proxies. *arXiv preprint arXiv:2606.06337*, 2026.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations (ICLR)*, 2024a.
- Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *USENIX Security Symposium*, 2024b.
- Chang Ma, Jun Ma, Kaixin Liu, Junzhe Li, Zhihao Mei, Guang Zhang, Zhan Wang, Yiqun Gu, Zhibo Xu, Zhijian Li, et al. Agentboard: An analytical evaluation board of multi-turn llm agents. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: A benchmark for general ai assistants. *arXiv preprint arXiv:2311.12983*, 2023.
- others. Compile, then page: Program-guided stack machine for llm agents. *arXiv preprint arXiv:2607.11346*, 2026a.
- others. Llm-as-code: Agentic programming. *arXiv preprint arXiv:2606.15874*, 2026b.
- others. Rel(ai)build: A deterministic control plane for llm coding agents. *arXiv preprint arXiv:2606.26924*, 2026c.
- others. Stream: Multi-tier llm inference middleware. *arXiv preprint arXiv:2606.13968*, 2026d.
- others. Xflow: Executable protocol programming for llm orchestration. *arXiv preprint arXiv:2606.14790*, 2026e.
- others. Dyserve: A workflow-aware serving layer for llm applications. *arXiv preprint arXiv:2607.02942*, 2027.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. Memgpt: Towards llms as operating systems. In *arXiv preprint arXiv:2310.08560*, 2023.
- Baolin Peng, Michel Galley, Pengcheng He, Hao Cheng, Yujia Xie, Yu Hu, Qiuyuan Huang, Lars Liden, Zhou Yu, Weizhu Chen, and Jianfeng Gao. Check your facts and try again: Improving large language models with external knowledge and automated feedback. *arXiv preprint arXiv:2302.12813*, 2023.
- Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, Ning Ding, Ganqu Cui, Zheni Zeng, Yufei Huang, Chaojun Xiao, Chi Han, et al. Tool learning with foundation models. *arXiv preprint arXiv:2304.08354*, 2023.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations (ICLR)*, 2024.
- Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, HyoJung Han, et al. The prompt report: A systematic survey of prompt engineering techniques. *arXiv preprint arXiv:2406.06608*, 2024.
- Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph E Gonzalez, and Ion Stoica. S-LoRA: Serving thousands of concurrent LoRA adapters. *arXiv preprint arXiv:2311.03285*, 2023.

- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Theodore R Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L Griffiths. Cognitive architectures for language agents. *arXiv preprint arXiv:2309.02427*, 2023.
- OmO Team. Oh-my-openagent (omo): A plugin-based agent operating system with team mode. <https://github.com/oh-my-openagent/omo>, 2026.
- Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *arXiv preprint arXiv:2308.11432*, 2023a.
- Lei Wang, Yihuai Xu, WanyuLan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023b.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023a.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023b.
- Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.
- Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.